

Digital Clock: Module Construction and Project Details.

Alasdair Nicol. S.N:1759586

1/5/2026

Contents

0.1	Project Setup	3
0.1.1	Backup	3
0.2	Assignment 1 Modules	3
0.2.1	Module 1: Seven Segment	4
0.2.2	Module 2: Up Down Counter	4
0.2.3	Module 3: Binary to BCD	5
0.2.4	Module 4: HMS Counter	6
0.2.5	Module 5: Mod N Counter	7
0.2.6	Module 6: Restartable Rate Generator	8
0.2.7	Module 7: Top Time Display (v1)	9
0.2.8	Module 8: Pulse Width Modulation Generator	10
0.2.9	Module 9: Rising Edge Detector	11
0.2.10	Module 10: Button Hold Detect	12
0.2.11	Module 11: Button Hold Pulse	13
0.2.12	Module 12: Button Auto Repeat	14
0.2.13	Module 13: Arming Latch	15
0.2.14	Module 14: Edit Mode Selector	16
0.2.15	Module 15: Editable Counter	17
0.2.16	Module 16: Key Synchroniser	18
0.2.17	Module 17: Decimal Display Driver	19
0.2.18	Module 18: User Top Watch V1	20
0.2.19	Module 19: User Top Watch V2	21
0.2.20	Module 20: User Top Watch V3	22
0.2.21	Module 21: User Top Watch V4	23
0.3	Assignment 2 Modules	24
0.3.1	Module 22: Cascade Counter	25
0.3.2	Module 23: Stopwatch Counter	25
0.3.3	Module 24: Snapshot Multiplexer	25
0.3.4	Module 25: Stopwatch Control	25
0.3.5	Module 26: User Top Stopwatch Version 1	25
0.3.6	Module 27: User Brightness Wrapper	25
0.3.7	Module 28: User Top Timepiece V1	26
0.3.8	Module 29: User Top Brightness Timepiece	26
0.4	Assignment 1 Questions	26
0.4.1	Report 1	26

0.4.2	Report 4	26
0.4.3	Report 5	26
0.4.4	Report 6	26
0.4.5	Report 7	28
0.4.6	Report 8	28
0.4.7	Report 9	28
0.4.8	Report 10	29
0.4.9	Report 11	29
0.4.10	Report 12	29
0.4.11	Report 13	29
0.4.12	Report 14	29
0.4.13	Report 15	30
0.4.14	Report 16	30
0.4.15	Report 17	30
0.4.16	Report 18	30
0.5	Assignment 2 Questions	31
0.5.1	Report 1	31
0.5.2	Report 2	31
0.5.3	Report 3	31
0.5.4	Report 4	31
0.5.5	Report 5	32
0.5.6	Report 6	32
0.5.7	Report 7	33
0.5.8	Report 8	33
0.5.9	Report 9	33
0.5.10	Report 10	33
0.5.11	Report 11	33
0.5.12	Report 12	33
0.5.13	Report 13	34
0.5.14	Report 14	34
0.5.15	Report 15	34
0.5.16	Report 16	34
0.5.17	Report 17	35
0.5.18	Report 18	35
0.5.19	Report 19	36
0.5.20	Report 20	36
0.5.21	Report 21	36

0.1 Project Setup

This section covers key information about the project settings, and backup settings for the project.

0.1.1 Backup

Document was backed up both locally and in an online repository using the git command-line tool. A backup was made at the end of each module design cycle and before closing the project workspace.

0.2 Assignment 1 Modules

This section covers the function and testing of each module in the assignment 1 portion of the project.

0.2.1 Module 1: Seven Segment

Waveform

The waveform in figure 1 demonstrates the correct 7f output (1111111) for the active low display when the blank signal is high. Otherwise, the output displays the correct bit encoding for the given input value.

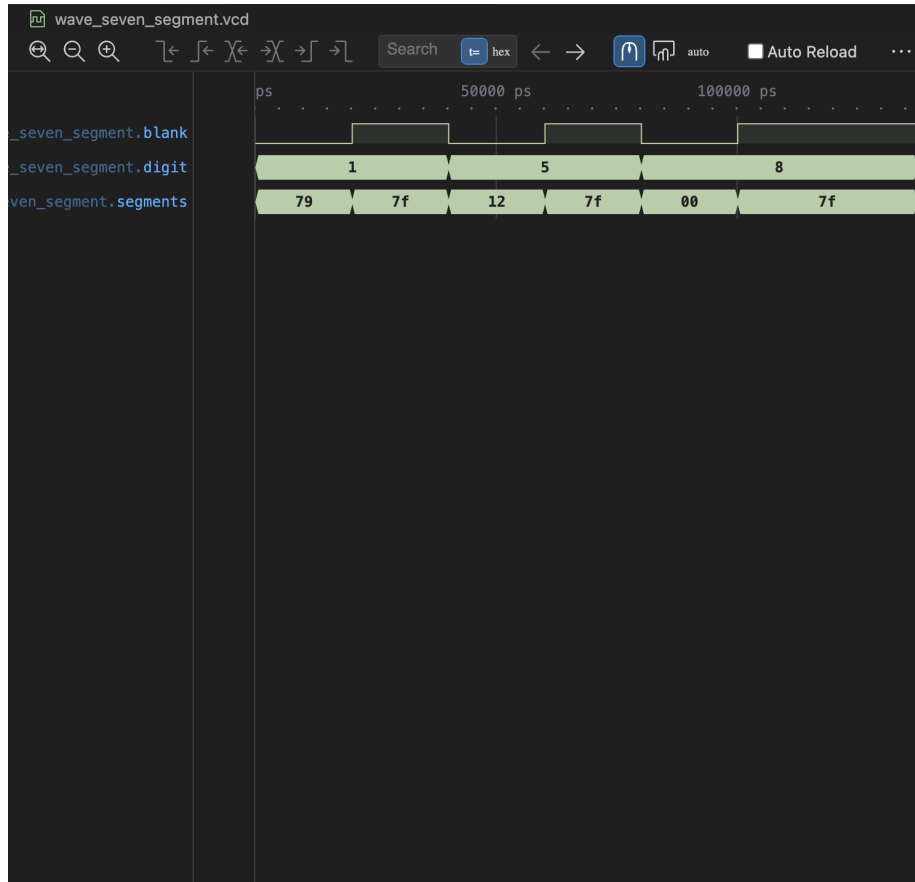


Figure 1: Waveform generated for the seven segment module.

0.2.2 Module 2: Up Down Counter

Waveform

When the enable signal is low, the counter correctly holds onto the current value of 0 (as seen in figure 2), before counting up in increments of one when both the enable pin and up pin are high. Furthermore, the timer correctly counts backwards with a unit of one at the next clock signal after the up signal goes low.

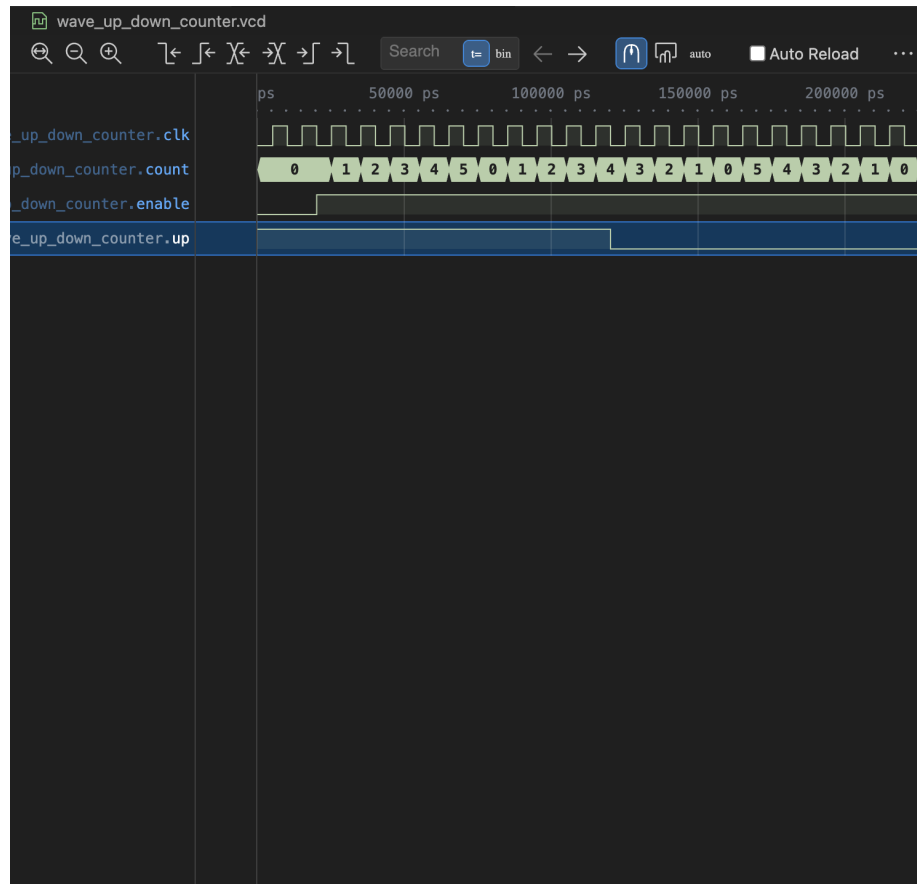


Figure 2: Waveform generated for the up down counter.

0.2.3 Module 3: Binary to BCD

Waveform

As can be seen with the bin signal of 63 in figure 3, the circuit correctly breaks down the hexadecimal 63(decimal 99) into the ones and tens components of 9 and 9.

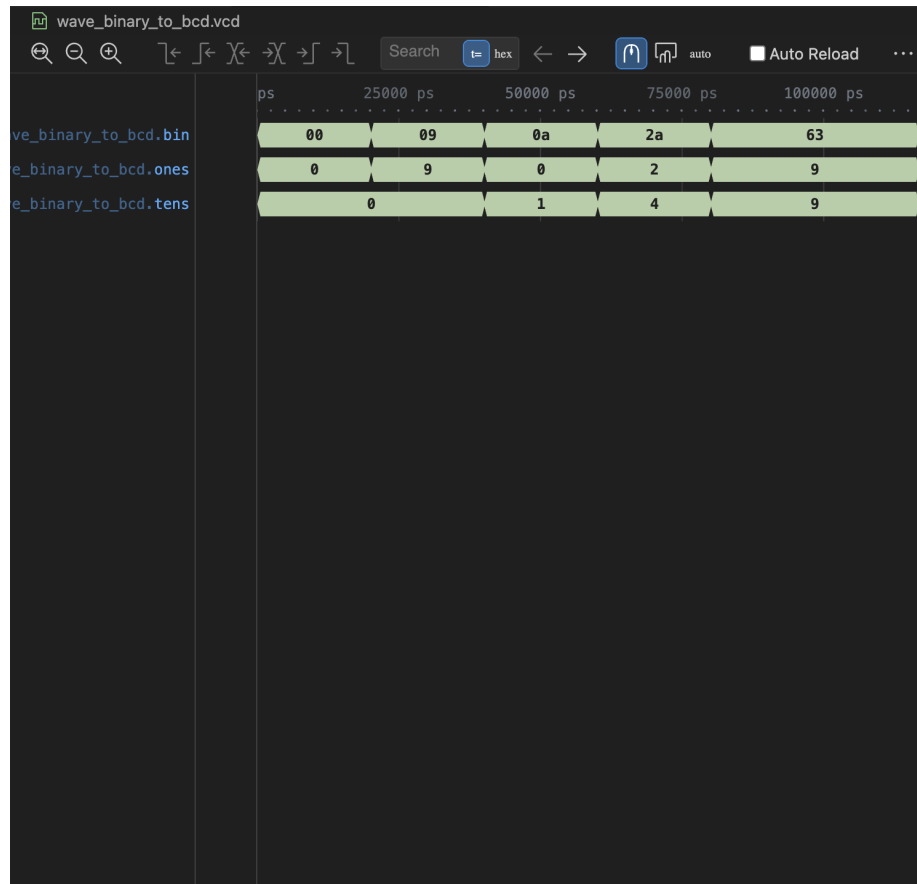


Figure 3: Waveform generated for the binary to bcd converter.

0.2.4 Module 4: HMS Counter

Waveform

The seconds signal of figure /reffig:hms successfully iterates to 4 on the rising edge of the rising edge of the clock, where it then switches to 0 and triggers an iteration in the minutes signal. This is the expected defined bahvaiour given the bit width and max count values used to test the circuit. The enable signal is also seen to hold the seconds signal at 0 from time 0 to the high enable signal.

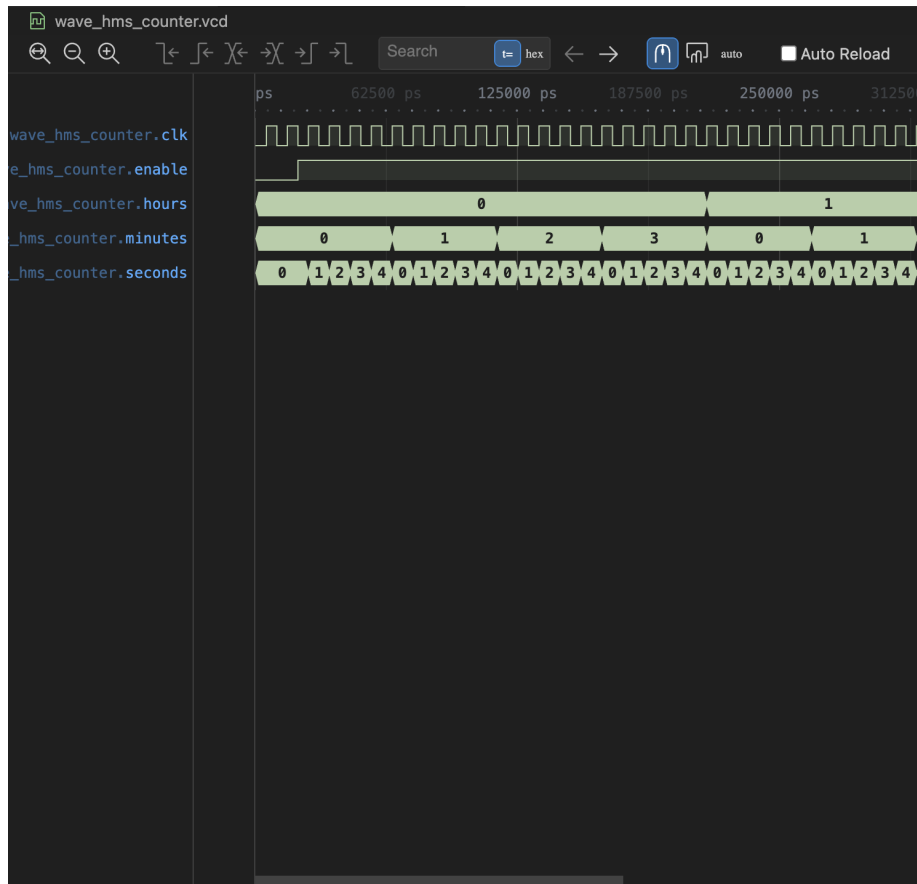


Figure 4: Waveform generated for the HMS counter.

0.2.5 Module 5: Mod N Counter

Waveform

Figure 5 demonstrates that the count variable correctly iterates by one on the rising clock edge only if the enable signal is high and the rst signal is low. At the low edge of the rst signal, the counter resets to 0 at the next rising edge of the clock. At the low edge of the enable signal the counter holds its previous value.

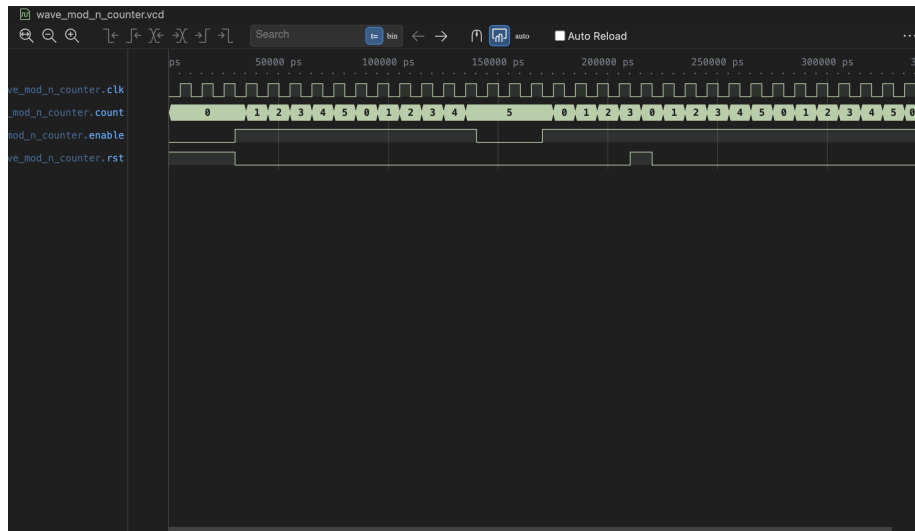


Figure 5: Waveform generated for the mod n counter.

0.2.6 Module 6: Restartable Rate Generator

Waveform

Figure 6 shows that the tick signal will pulse for one clock cycle when the counter reaches it's maximum value defined by `CYCLE_COUNT`. The running signal correctly follows the run signal with a clock cycle delay. Furthermore, the counter resets each clock cycle that the running signal is low.

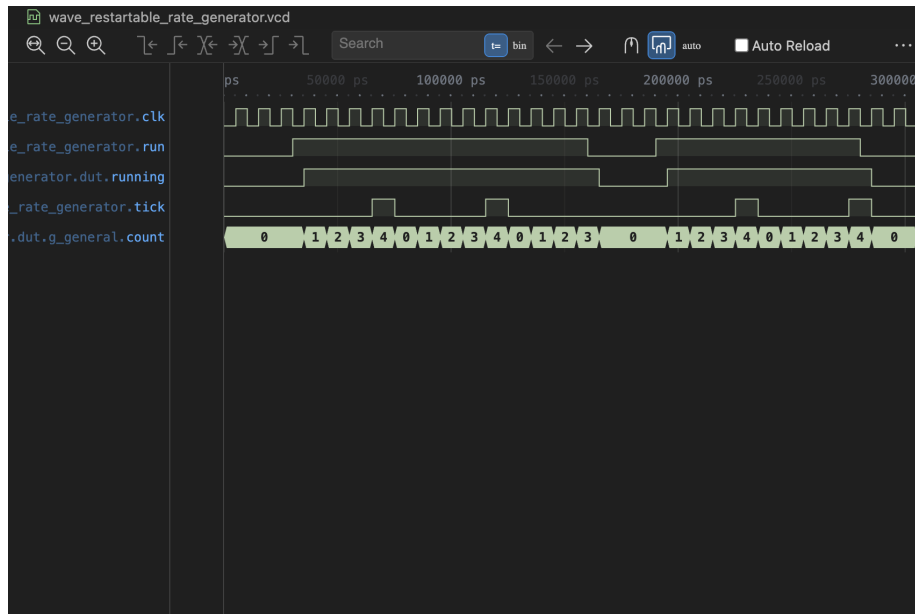


Figure 6: Waveform generated for restartable rate generator.

0.2.7 Module 7: Top Time Display (v1)

Waveform

Figure 7 demonstrates a initial pause as the input switch specifies the longest second period. After the timer is switched to the faster clock mode, the seconds correct increments by one with the hexadecimal output directly matching the seconds input. The seconds count reaches 59 and rolls over, incrementing the minute count by 1 which is mirrored in the hexadecimal output signal. This behaviour continues through the test.

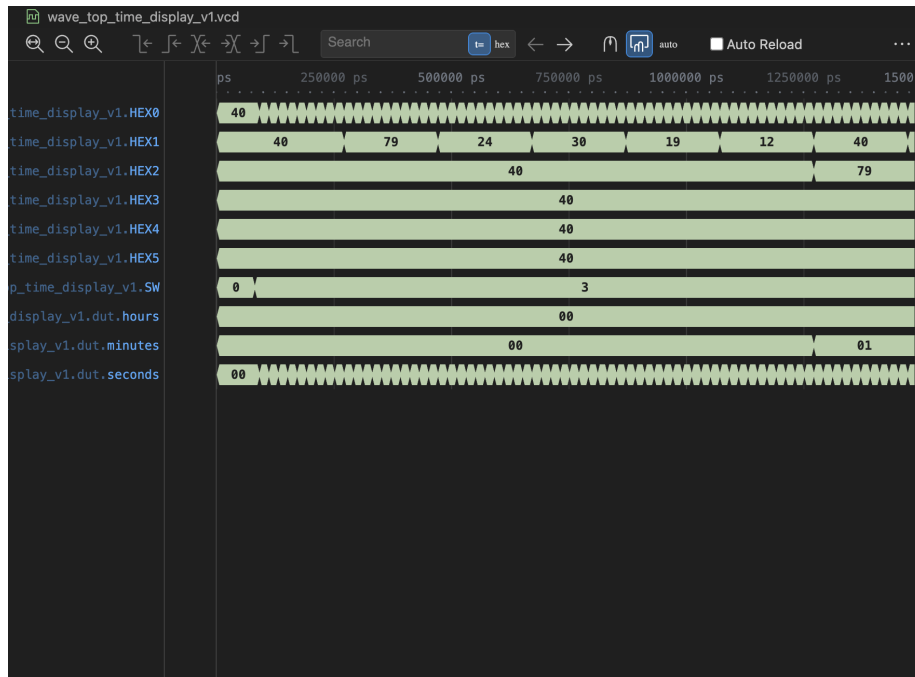


Figure 7: Waveform generated for the first top time display.

0.2.8 Module 8: Pulse Width Modulation Generator

Waveform

Figure 8 shows that the count signal increments to the cycle number, and correctly holds at 0 when the rst signal is high. The PWM signal holds at 1 for the specified duty cycle, and remains at 0 for the rest of the period.

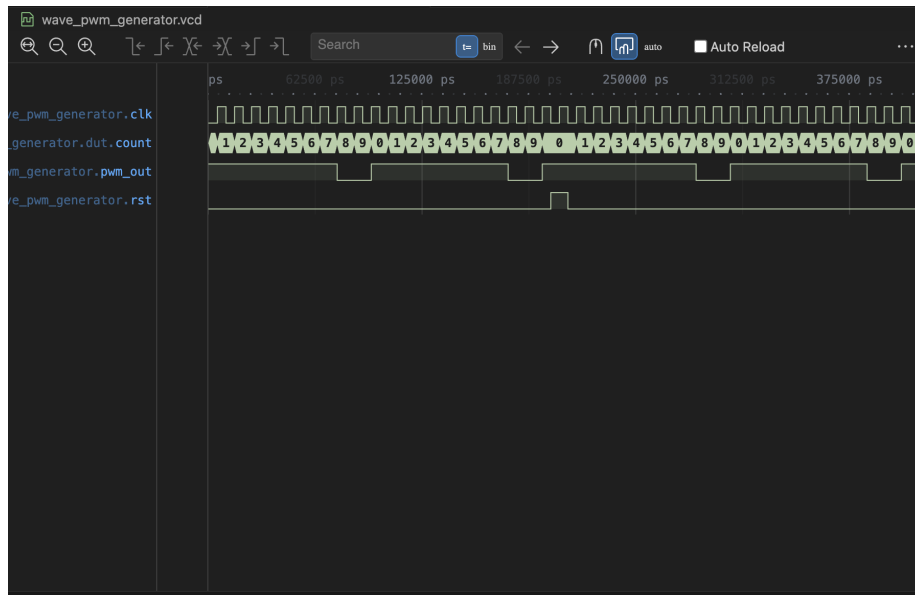


Figure 8: Waveform generated for pwm generator.

0.2.9 Module 9: Rising Edge Detector

Waveform

Figure 9 demonstrates a clock cycle wide pulse on the pwm_out signal when the sig_in signal goes high and the clock pulses. If the pulse occurs within a clock cycle, no output occurs as expected.

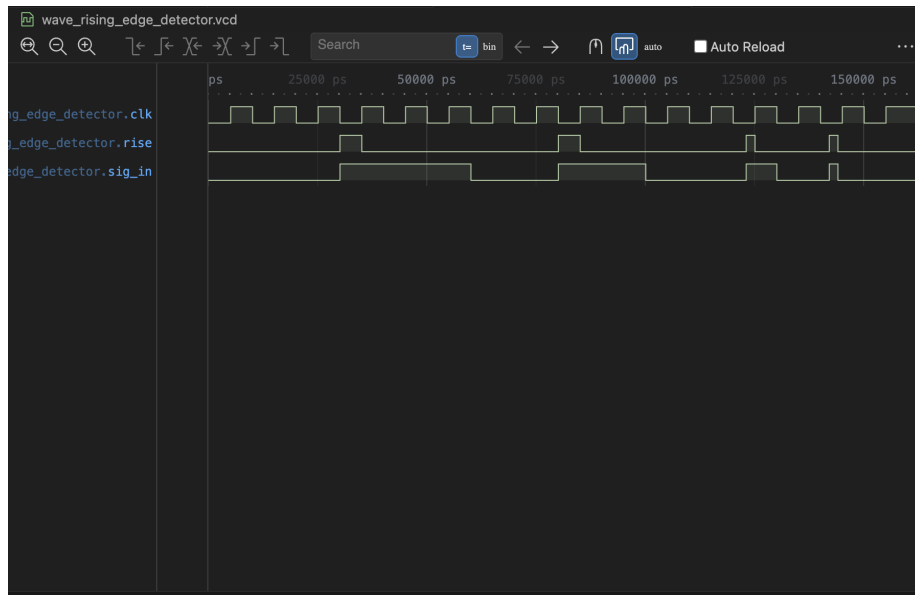


Figure 9: Waveform generated for rising edge detector.

0.2.10 Module 10: Button Hold Detect

Waveform

Figure 10 shows that the counter resets every cycle until the button is pressed, for which it begins to count at the next clock cycle while the button remains pressed. If the counter reaches the specified number of cycles, the held signal will go high. This disables the counter and holds the held signal high until the button is released and the counter is reset.

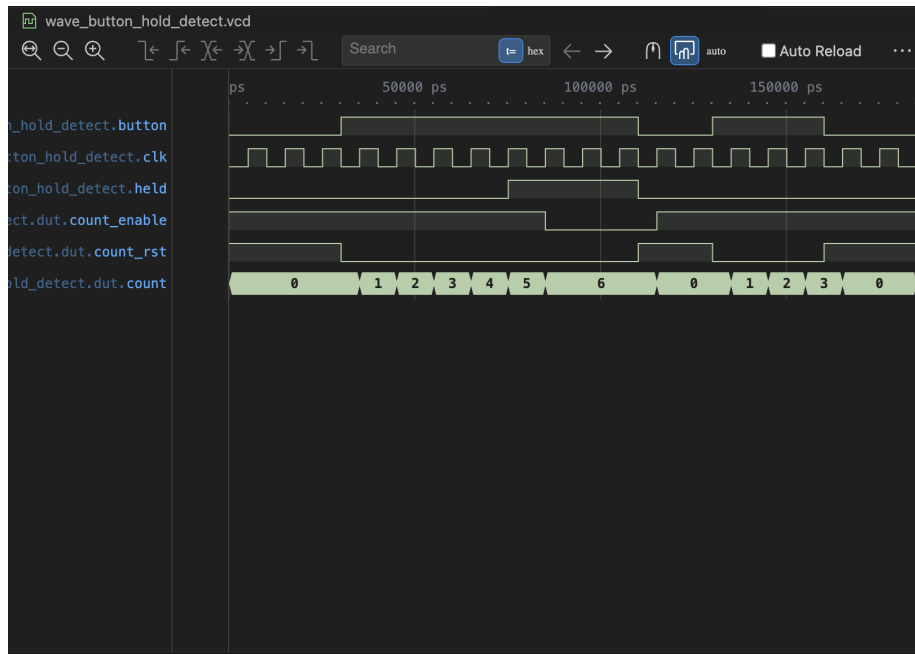


Figure 10: Waveform generated for the button hold detector.

0.2.11 Module 11: Button Hold Pulse

Waveform

Figure 11 shows a pulse signal one clock cycle wide triggered only after the button signal has been held high for a specified number of clock cycles. No pulse occurs if the button is not held down for the specified number of clock cycles, and the pulse signal does not remain high as the previous module specified.

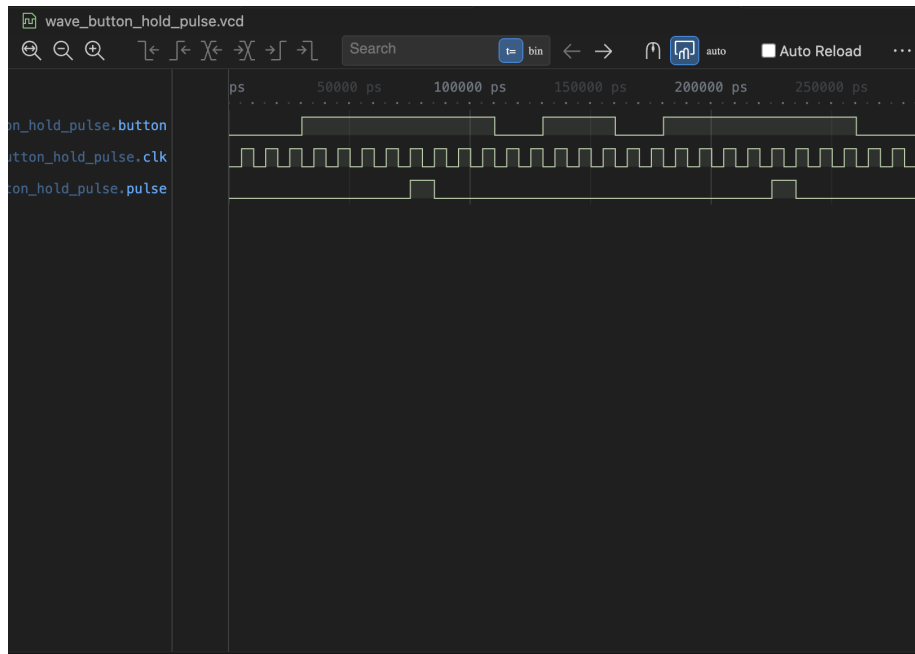


Figure 11: Waveform generated for the button hold pulser.

0.2.12 Module 12: Button Auto Repeat

Waveform

The pulse signal in figure 12 demonstrates a brief pulse cutoff by the next clock rising edge as the button signal goes high. The counter then begins in the `button_hold_detect` module which produces a high signal after 7 clock pulses which feeds the run signal of the rate generator. The rate generator produces a periodic pulse on the pulse wire until the button is released.

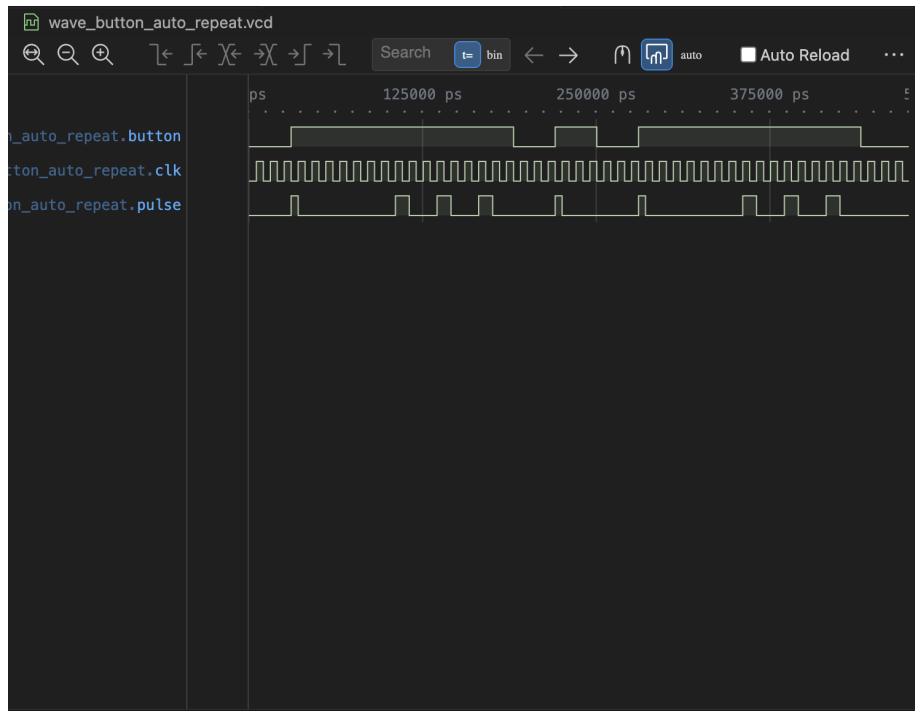


Figure 12: Waveform generated for the button auto repeat module.

0.2.13 Module 13: Arming Latch

Waveform

The armed signal in figure ?? goes high at the next clock cycle after the arm signal has gone high, and the disarm signal is low. The armed signal stays high until the disarm signal is pulled high. If both the arm and disarm signals go high, the disarm signal takes priority.

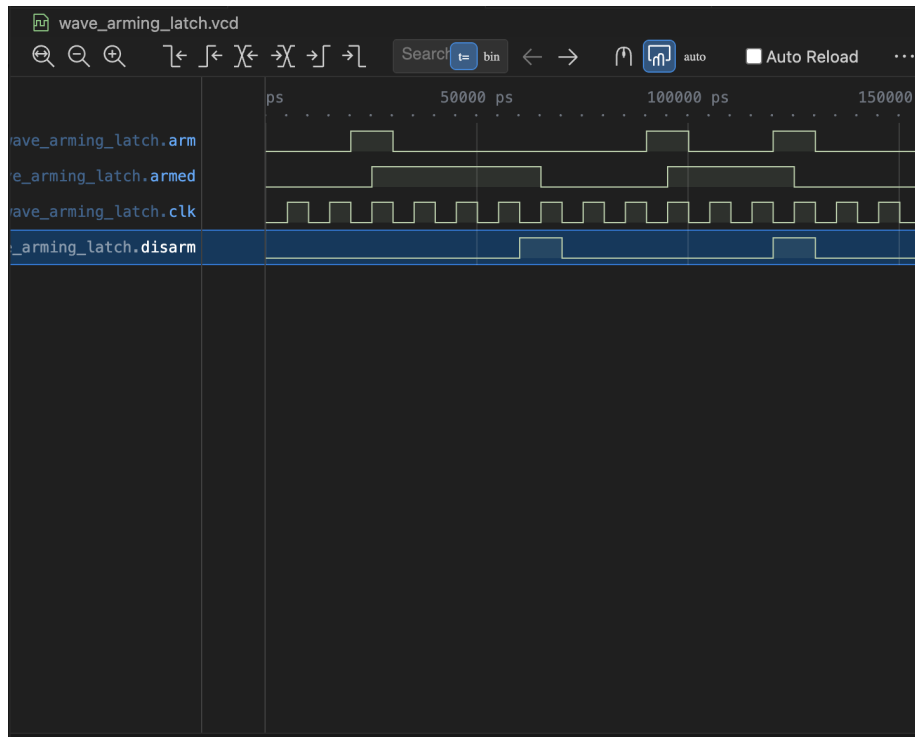


Figure 13: Waveform generated for the armed latch.

0.2.14 Module 14: Edit Mode Selector

Waveform

Figure 14 demonstrates that the button must remain pressed for the specified number of clock cycles to enter seconds mode. Later brief presses shift the state into minutes mode, and then hours mode before resetting the counter and disarming the system.

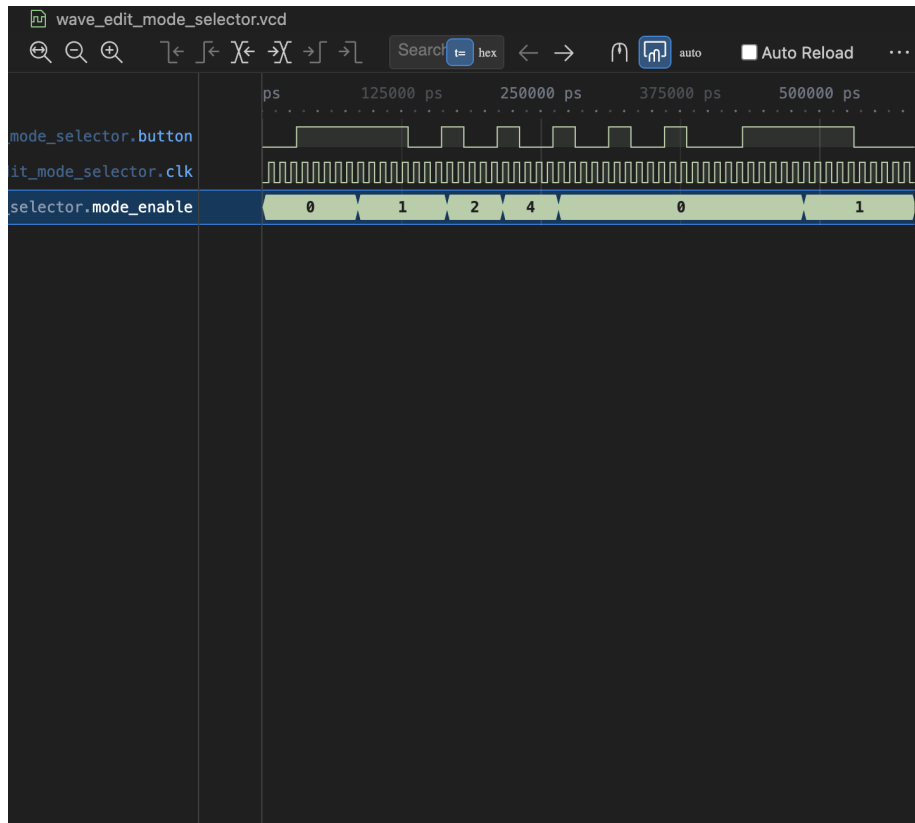


Figure 14: Waveform generated for the edit mode selector module.

0.2.15 Module 15: Editable Counter

Waveform

The counter increments with tick when edit mode is off as seen in Figure 15. Once the edit mode signal is pulled high, the inc signal is pulled high which initially increments the counter, before the tick signal continues to increment the counter. When dec mode is enabled, the counter continually counts down before wrapping around to the maximum value. When both the inc and dec signals are high, the counter does nothing. Once the edit mode signal goes low, the counter continues to increment with the tick signal.

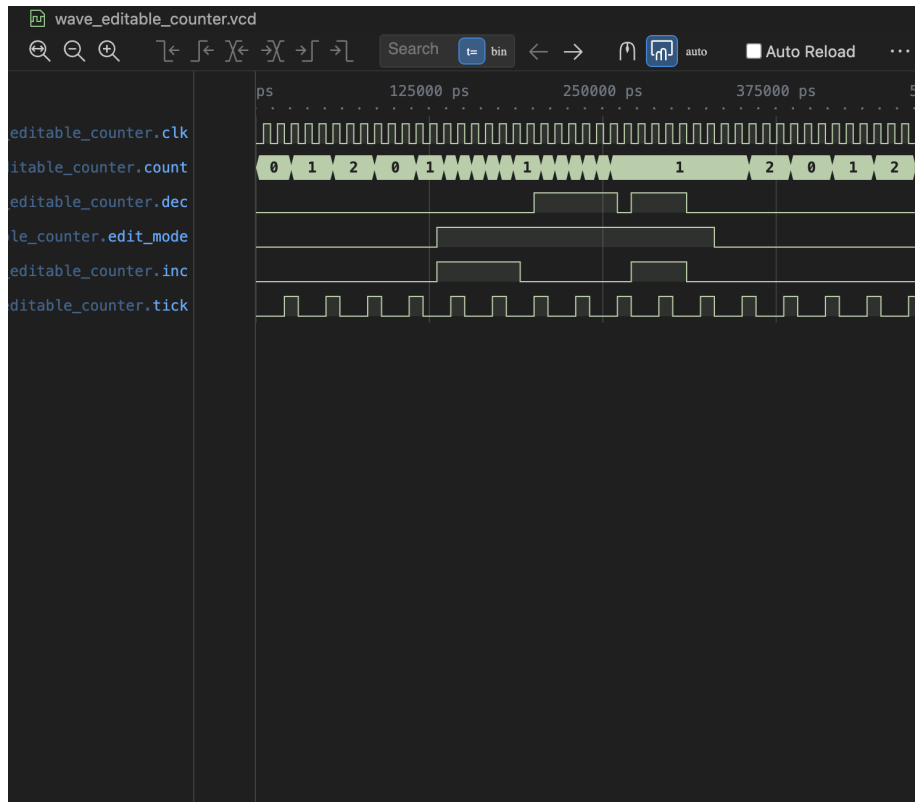


Figure 15: Waveform generated for the editable counter.

0.2.16 Module 16: Key Synchroniser

Waveform

The key state is inverted and stored in the key register at the next clock rising edge as seen in figure 16. On the next rising edge the inverted key state is transferred to the key_sync_next register which is directly linked to the key_sync output. The waveform also demonstrates that the button press must coincide with a clock pulse to be read.

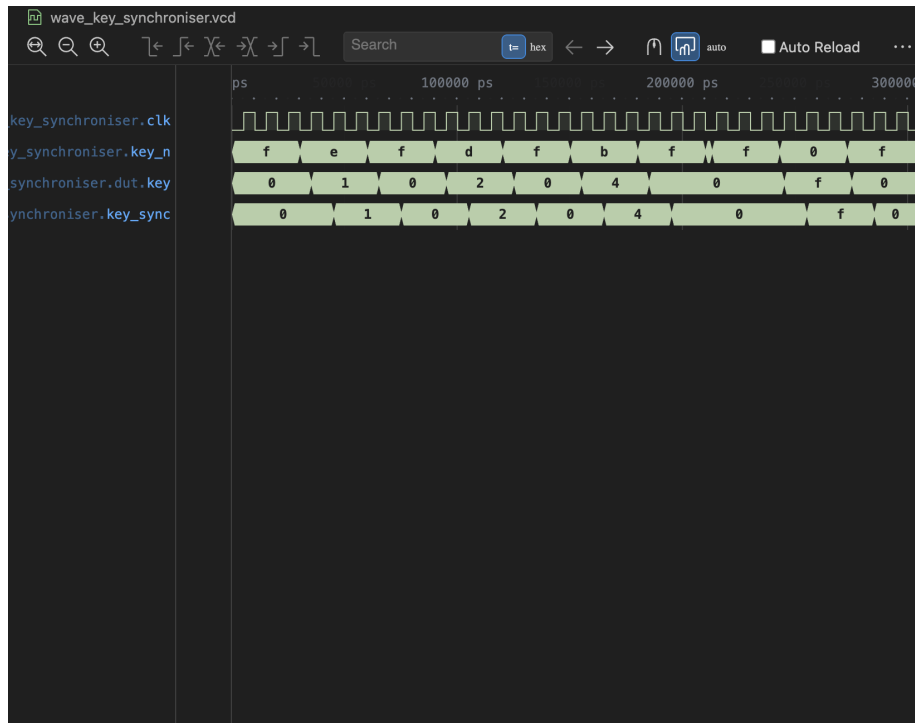


Figure 16: Waveform generated for the key synchroniser.

0.2.17 Module 17: Decimal Display Driver

Waveform

The hexadecimal outputs correctly follow the three value variables in figure 17. As the blank signals are pulled high the hexadecimal digits connected to the blank signal display '7F' which is the correct hexadecimal to describe all segments as off for an active low hex display.

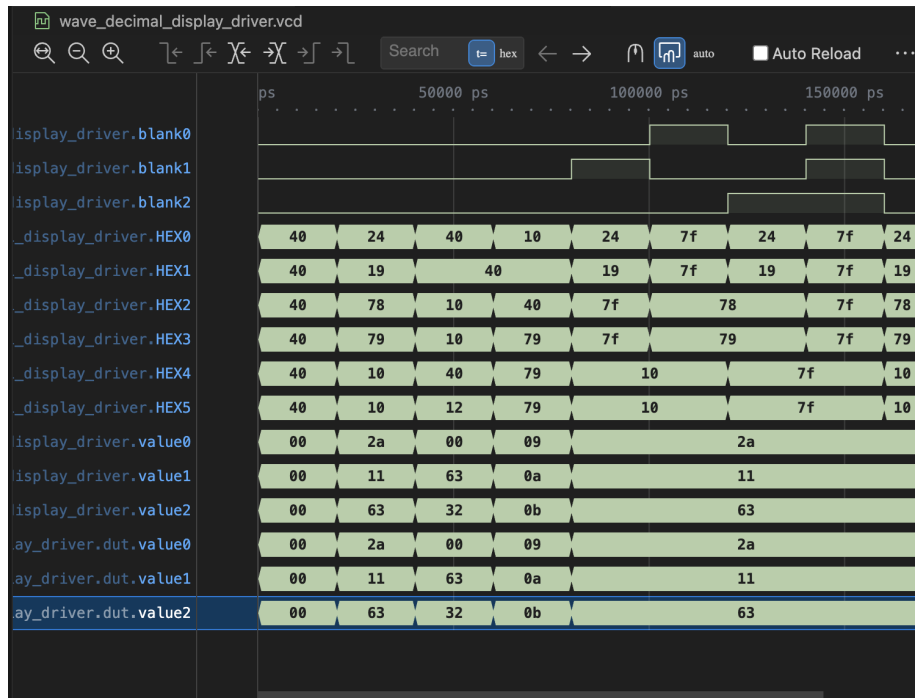


Figure 17: Waveform generated for the decimal display driver.

0.2.18 Module 18: User Top Watch V1

Waveform

Note that signals temporarily assigned to low for this simpler module have been excluded for simplicity. The 'disp' outputs demonstrate the incrementing behavior of the seconds counter which reaches a value of 59 before incrementing the minutes timer on the same clock cycle (as seen in 18). This behavior is repeated as the minute_disp signal successfully increments to 2.

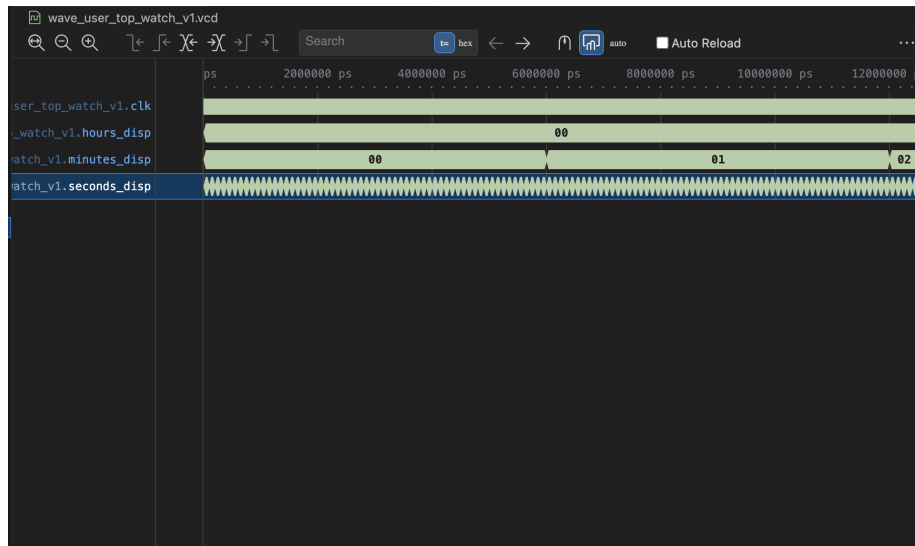


Figure 18: Waveform generated for the first user top watch module.

0.2.19 Module 19: User Top Watch V2

Waveform

Note that signals not being tested have been deliberately excluded for simplicity. The initial state of figure 19 has the blank signals are held low and button[3] (8 in decimal) has not been pressed. Button[3] is the held for 50 cycles which triggers the edit mode, seen by the switch in the mode_enable signal. This causes the PWM signal to pull the seconds_blank signal low and high accordingly. Two more brief button presses switch the mode to minute and hour edit modes, where the minute and hour blanking signals follow the PWM signal accordingly.

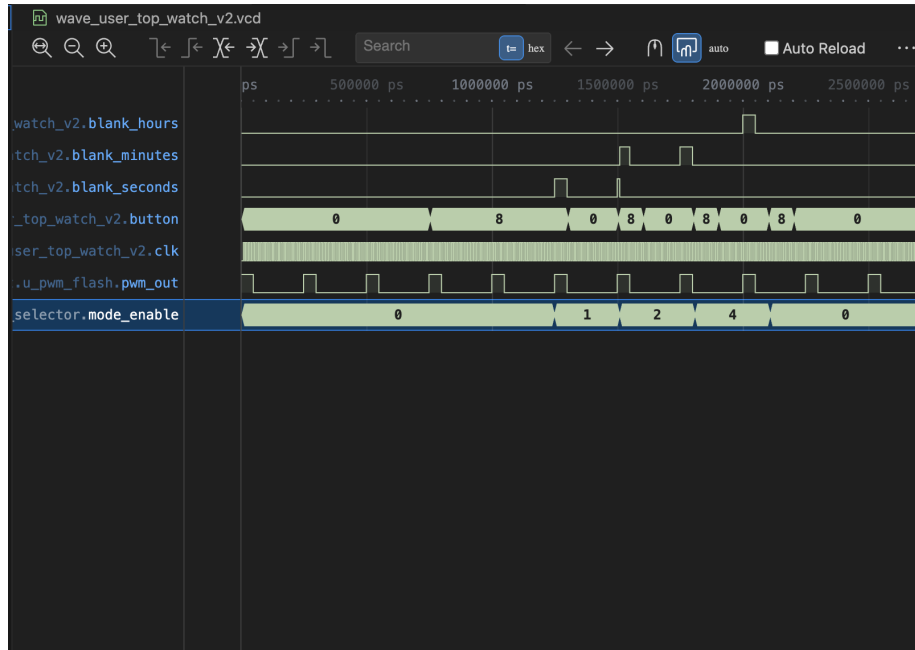


Figure 19: Waveform generated for the second user top watch module.

0.2.20 Module 20: User Top Watch V3

Waveform

Note that signals not being tested have been deliberately excluded for simplicity. The initial state of figure 20 has the seconds counter incrementing by one unit every second (or simulated second defined by `CYCLES_PER_SECOND`). This does not change until `button[3]` is held long enough to enter edit mode (decimal 8). As the increment button (decimal 2) is pressed, the seconds count increments by one unit. If the increment button is held for 0.5s (as defined by the cycles per second) the seconds counter begins rapidly increasing. It is further demonstrated that a single pulse from the decrement button (decimal 1) decreases the second count by one. This behaviour is then tested and verified for the minutes and hours counters.

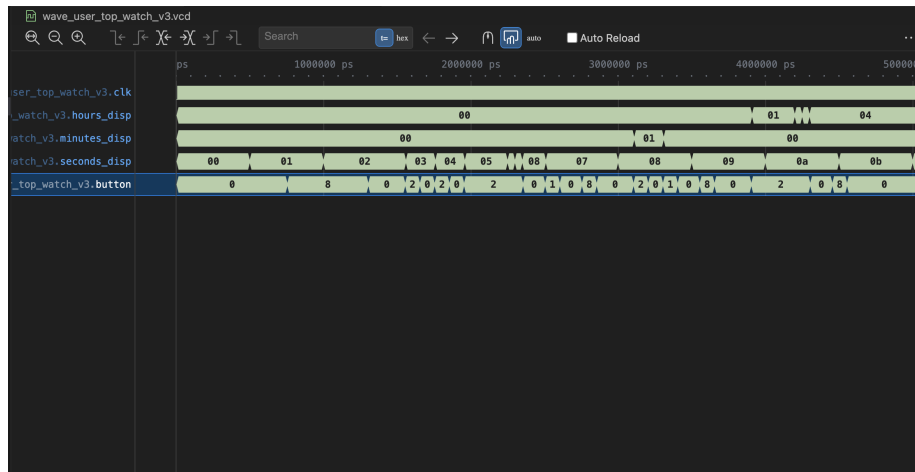


Figure 20: Waveform generated for the third user top watch module.

0.2.21 Module 21: User Top Watch V4

Waveform

Note that signals not being tested have been deliberately excluded for simplicity. The initial state of figure 21 has no input, which results in the seconds counter incrementing normally. Button[3] (decimal 8 on the button signal) is held down to bring the mode_enable signal to 1 (seconds edit mode). In this state, the seconds counter no longer increments, which is the expected behavior for the v4 additions. Once button[3] has been pressed again, the mode_enable signal is set to 2 bringing the system out of seconds edit mode. At the next seconds clock pulse, the seconds counter increments again demonstrating that the defined behavior from the v1 to v3 has not been altered.

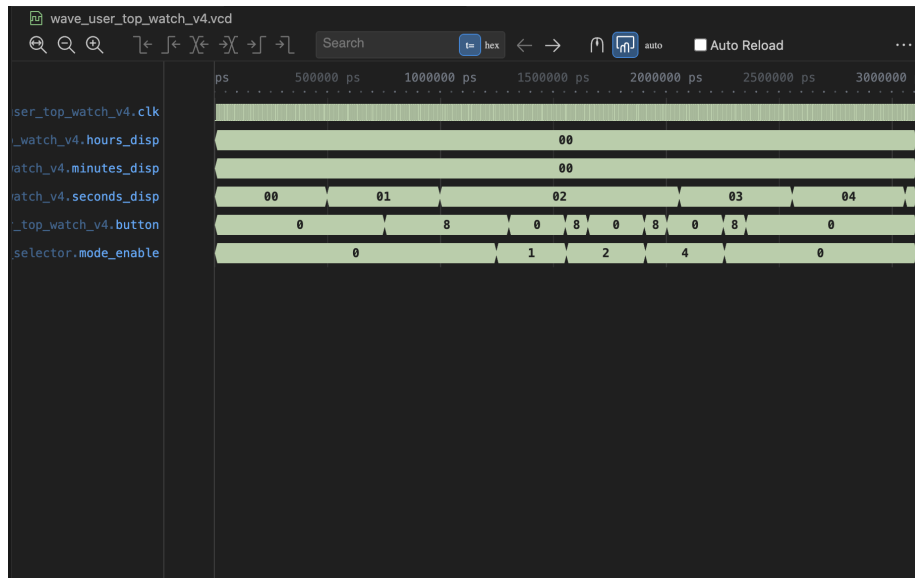


Figure 21: Waveform generated for the fourth user top watch module.

0.3 Assignment 2 Modules

This section covers the function and testing of each module in the assignment 2 portion of the project (The rest of the project).

0.3.1 Module 22: Cascade Counter

Description

The cascade counter module is a generalised HMS counter with no edit function. As it is only required for the stopwatch, it must only count up. Editing the time would beat the point of the stopwatch (ex. 3s 100m sprint times).

0.3.2 Module 23: Stopwatch Counter

Description

The stopwatch counter is a specialised implementation of the cascade counter that counts in minutes, seconds and centiseconds and is driven by a centisecond restartable rate generator.

0.3.3 Module 24: Snapshot Multiplexer

Description

The multiplexer takes in a register of signals, and stores the current input signal in an internal register. Under normal operation the input is linked combinationally with the output. When the hold signal is pulled high the module stops updating the internal register and the output reflects the internal register.

0.3.4 Module 25: Stopwatch Control

Description

This module defines a moore FSM for the different stopwatch functions to fit the given specification (listed below).

```
// Pressing start stop toggles stopwatch b/t running and stopped
// If running with live display, lap freezes the display
// If running with a frozen display, lap unfreezes the display
// If stopped with a frozen display, lap unfreezes the display (no rst)
// While stopped with a live display, lap resets time to 0
// If both buttons are pressed, then both are ignored
```

0.3.5 Module 26: User Top Stopwatch Version 1

This module is prototype. The test cases were successful. Further documentation will be saved for the final module.

0.3.6 Module 27: User Brightness Wrapper

Module sucsefully modulates blank signals according to PWM specifications. All test cases in both SBY and Pytest have been passed.

0.3.7 Module 28: User Top Timepiece V1

Module successfully instantiates and operates modules. Module passed required testing.

0.3.8 Module 29: User Top Brightness Timepiece

Module successfully instantiates and operates modules. Module passed required testing.

0.4 Assignment 1 Questions

0.4.1 Report 1

Blank is a 1 bit wide logic signal, while ACTIVE LOW is defined as an integer, which is a multi-bit data type (32 by default) which will throw a warning if truncated.

0.4.2 Report 4

BCD is a method of encoding decimal bits in binary that reserves a number of bits (usually 4 for 0-9) for each decimal digit, before concatenating all of the digits into a single register. Storing the digits individually provides greater accuracy at high counts, and helps to avoid rounding errors making it a common tool in precision counters.

0.4.3 Report 5

The code uses the clog2 function, which takes the ceiling of the log2 operation on the number of cycles. This answers the question of "how many powers of 2 (2^n) do I need to have a number greater than the cycle count?". As the maximum number stored in a set of bits also expands such that the maximum storeable number is 2^n where n is the result of clog2, this statement produces the minimum number of bits required to store the cycle count in a register.

0.4.4 Report 6

The code used for the testbench was stored in waverestartable_rate_generator_one.v and produced the output in wave_restartable_rate_generator_one.vcd. The code is a direct modification of wave_restartable_rate_generator_one.v, using a CYCLE_COUNT of 1.

```
// Wave restartable rate generator testbench for max cycle of 1 case
// Based of wave_restartable_rate_generator.v written by Jonathan Manton
//
`timescale 1ns / 1ps

module wave_restartable_rate_generator_one;
    reg clk = 0;
    reg run = 0;
    wire tick;
```

```

restartable_rate_generator #(
    .CYCLE_COUNT(1)
) dut (
    .clk (clk),
    .run (run),
    .tick(tick)
);

always #5 clk = ~clk;

initial begin
    $dumpfile("wave_restartable_rate_generator_one.vcd");
    $dumpvars(0, wave_restartable_rate_generator_one);

    // Run: tick fires every CYCLE_COUNT clocks
    #30;
    run = 1;
    #130;

    // Disable mid-cycle: counter resets, no stray tick on re-enable
    run = 0;
    #30;
    run = 1;
    #90;

    run = 0;
    #20 $finish;
end
endmodule

```

The code produced the wave table in figure 22.

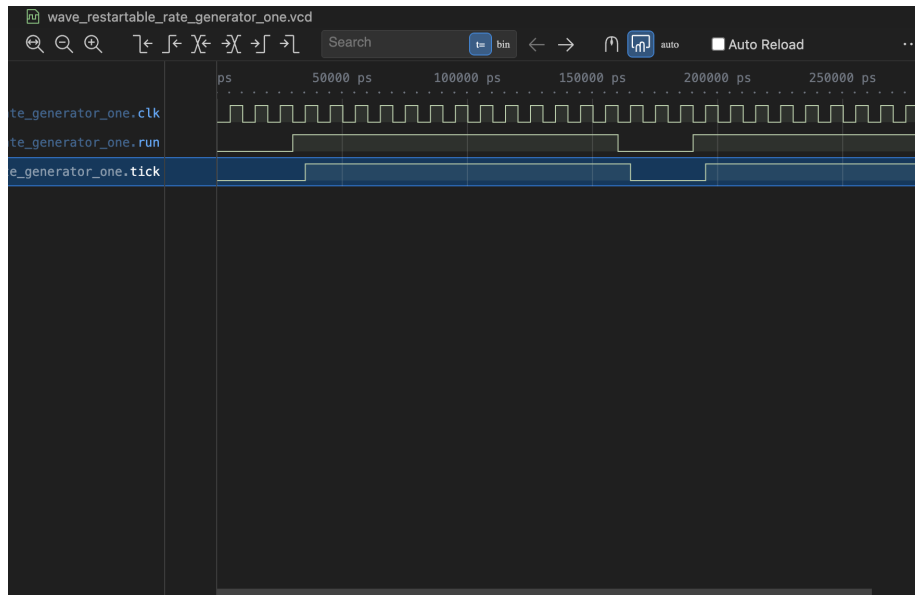


Figure 22: Waveform generated for the mod n counter.

The tick signal remains high while after the run variable is high (and has settled), which is the correct special specified behaviour for the case where `CYCLE_COUNT = 1`.

0.4.5 Report 7

The module name can be lengthened to pulse width modulation generator, which describes the modules real purpose. The module produces a signal with a specified on and off time over a period, usually to controll the amplitude of a connected device (as the average signal can be raised or lowered depending on how long the signal is on or off, otherwise known as the duty cycle). This allows for specific voltage control without using a digital to analog converter.

0.4.6 Report 8

It allows for the rst signal to be used as a form of enable function, keeping the counter at 0 while it is high. In the PWM generator module this provides an easy way to pause the counter and reset the PWM cycle at the same time.

0.4.7 Report 9

As the output is a function of the current state, feeding the next state logic with the output logic is still equivelant to just processing the current state using more complex next state logic, therefore the system is still a Moore FSM.

0.4.8 Report 10

Figure 11 contains the waveform produced by the module. The module utilises the `button_hold_detect` module which pulls the output high once the input button has been pressed for a specified number of clock cycles. The continuous high output of the detect module is wired to the rising edge detector module, which converts a rising edge into a once cycle wide pulse signal, which is the desired output for this module.

0.4.9 Report 11

While the `rising_edge_detector` module is a mealy FSM (as the high / pulse output is dependant on a rising edge input), it is implemented as part of the next state logic in this module. When looking at the inputs and outputs of the `button_hold_pulse` module, the button and clk signals only drive the two sub-modules, and are not used to assign an output directly, which makes this module a Moore FSM rather than a Mealy.

0.4.10 Report 12

STILL NEED TO COMPLETE

0.4.11 Report 13

The two types are handled differently at a compiler level (logic being considered a variable along with int, bit etc. while wire is a net data type). In this case, the wire net type specifies that a signal can be driven by multiple sources, therefore the wire net type (that automatically implies a logic type) allows the logic signals to be driven by multiple sources in this code.

0.4.12 Report 14

Proper testing of the system on the board would take at least 48 hours to confirm two correct cycles of the board. This is assuming that there was no debugging required when compiling the code in Quartus. In reality each iteration could take 3 days to properly test along with lab equipment to capture the behavior of the board on camera for the complete test time. This would be completely impractical to complete for every version of the software.

The automated test suite allows the programmer to fail early and inexpensively as the test suite can catch bugs even at the basic testing phase of the project (as long as the behavior has been defined prior to testing). By catching errors at the first stage of the project, the programmer can be confident that any errors encountered in the next stage are a result of the changes, and not the original code. This reduces the volume of code a programmer has to search through for a bug making big fixes in subsequent versions much quicker.

The test would take significantly longer to run for the same time period as there are more clock cycles to consider, and therefore more calculations that need to be completed

to test the modules (note that this does not apply for step based testing, which will simply complete faster). With this in mind, it is important to write testable code with opportunities to redefine clock cycles in order to quickly simulate the modules.

0.4.13 Report 15

Relevant waveforms have been chosen for the top timer report sections. to verify this, see sections 0.2.18 to 0.2.21. Relevant waveforms include only waveforms of direct effect or of direct consequence of the elements being tested.

0.4.14 Report 16

Assuming reset refers to the run function of the restartable rate generator, the design produces on one continuous reset for each time that the system is brought into the seconds editing state. This is not a problem as the run port is defined to hold the rate generator at it's current state if pulled low, therefore there is no need for multiple resets and adjustments of the rate generator keep the value correct without incrementing.

0.4.15 Report 17

It was very rare in the labs to achieve the correct function on the first test of module. Often details like next state logic or even variable names were mixed up which forced another 5 minute recompile cycle. The Visual Studio Code environment combined with pytest is a much more efficient method to program the modules as the compiler is quick, and the quality of life features such as real time linting make the development process significantly quicker. Not to mention the improved module accuracy when using automated test suites that catch small details such as clock cycle misalignment. For these reasons, I much prefer working in the Visual Studio Code environment.

0.4.16 Report 18

A deviation value of 50PPM means that at it's absolute maximum, the clock can deviate by 50 clock cycles for every 1 million clock cycles. Equation (1) is a calculation displaying the absolute maximum number of clock cycles out the De1-soc can become over a 24 hour period.

$$(50PPM) \times 50(MHz) \times (24 \times 60^2) = 216000000(cycles) \quad (1)$$

Equation (2) converts the clock cycles into seconds.

$$\frac{216000000(cycles)}{50,000,000(MHz)} = 4.32(S) \quad (2)$$

Therefore the maximum drift is 4.32 seconds. Note that this maximum drift is an absolute worse case scenario. In most cases, the clock will drift both forward and backward, resulting in a fraction of the drift per day.

0.5 Assignment 2 Questions

0.5.1 Report 1

It would also be possible to alter every instance of the counter by surrounding by adding a new reset function, or branching the project off completely to create a new version. Both of these options are far more impractical than the ones mentioned by the question. As the module is already referenced many times in code, altering the original file would require re-writing many other modules, which would all have to be retested and verified. The thin wrapper approach may work, but would be unnecessary given that all of the previously written code will refer to the original module while all the new code will refer to the new module. There is no time or resources saved when compared to copying the file and renaming it. Therefore it would be best to copy the module and rename it, altering only the new file.

0.5.2 Report 2

The assertions cover all reasonable behavior whether explicitly such as the parameter checks or implicitly, such as the maximum value being verified as part of the wrapping checks. With this in mind the verification must be run long enough (multiple full count cycles for both the count up and count down functions) in order to check that all conditions are met (eg. wrapping conditions don't happen every clock cycle).

0.5.3 Report 3

To verify the module for all values of MAX, the module must be checked with formal mathematical verification. Mathematical proof is the only method that can prove a process true for all input values rather than for specific test runs.

0.5.4 Report 4

Figure 23 demonstrates that an invalid value of 62 was used as an initial state for the test. This prevented the counter from incrementing once it was enabled therefore failing the count increment assertion on line 63 of the assertion file. By restoring the count $\leq$ MAX assertion, 62 would be considered an invalid starting case therefore would not constitute a valid base case for a formal proof, invalidating the failure.

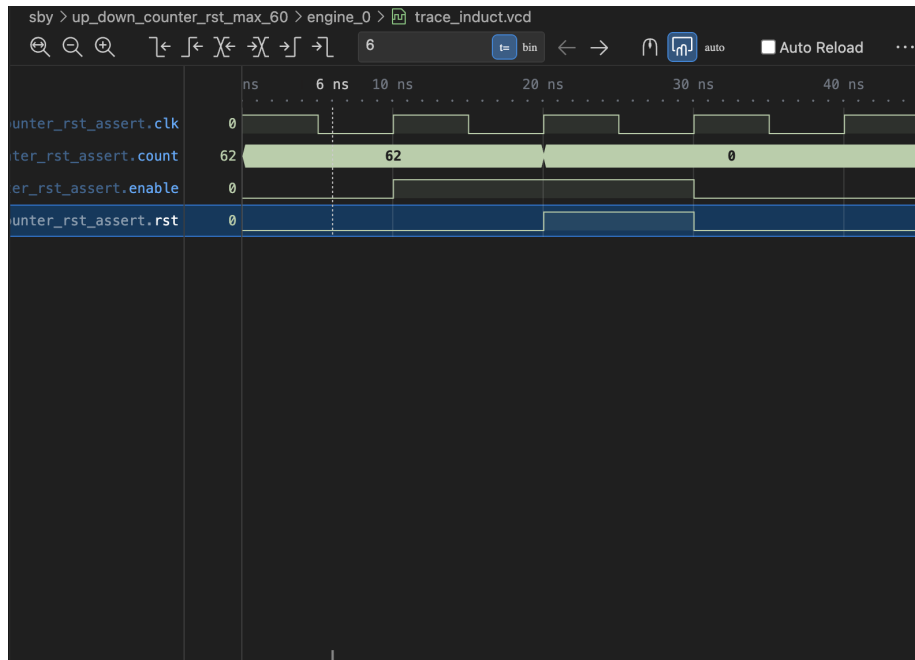


Figure 23: Waveform generated as a counterexample to formal verification.

0.5.5 Report 5

YET TO BE COMPLETED. WILL DO ONCE I'VE FINALISED THE DESIGN.

0.5.6 Report 6

At its most basic function, the stopwatch is a series of cascading counters for the seconds, minutes and hours with a secondary counter to track the laps. This base layer is responsible for basic counting functions and registers.

The next layer connects the seconds, minutes and hours timers together with a rate generator set to time seconds. This is supplemented with the correct rollover logic. This layer is responsible for correct timekeeping.

The next layer handles the lap function, taking snapshots of the time and storing them in the correct register. This layer is responsible for the lap logic.

The next layer handles the stop, start and lap logic and connects these button inputs to ports such as reset and enable. This layer is responsible for the correct input logic and timer function.

The top layer handles decimal to display conversions and display features. This layer is responsible for displaying information in a readable and useful way.

0.5.7 Report 7

YET TO BE COMPLETED. REQUIRES DRAWING.

0.5.8 Report 8

The enable signal of the counter is decided using the combinational logic `tick && enable`. STILL NEEDS TO BE FILLED OUT, QUESTION NEEDS CLARIFICATION.

0.5.9 Report 9

YET TO BE COMPLETED. REQUIRES DRAWING.

0.5.10 Report 10

YET TO BE COMPLETED. REQUIRES DRAWING.

0.5.11 Report 11

The borrow complement signal is generated with a NAND gate, of which the inputs are: NOT down, NOT output A, NOT output B, NOT output c and NOT output D. The BO signal will only turn off when the down function is off, and the counter is at 0. As the borrow complement signal represents a counter underflow (to trigger another chained counter IC to increase the number of counter registers), the schematic is saying that the BO (borrow) complement signal should only go low when the down signal is not high, and the counter is at 0. As we are interested in the borrow signal not the borrow complement signal, then we are looking for the inverse of this behavior.

0.5.12 Report 12

The first section implemented was the basic countdown and cascade counting feature set. This involved instantiating a restartable rate generator and three editable countdown modules where the borrow signal was used to tick the subsequent counter module. Next, the states were defined as local parameters and the next state logic was implemented. This also includes instantiating an edit mode selector, as the output was required in the next state logic.

The next step involved instantiating a rising edge detector for the state change button (`button[0]`) and two button auto repeats for the incrementing and decrementing buttons. These were linked to the counter modules.

Finally, with a range of modules instantiated, the assignments were described at the bottom of the module, most of which were copy pasted from the `user_top_watch_v4`.

0.5.13 Report 13

The states were chosen to be Stopped, Running and Set, with minute set, hour set and second set being sub-states of the Set state. As the states were decided early in the design of the module, a simple binary state encoding was chosen to mirror forms seen earlier in the course. This has the added advantage of minimising flip flops.

From the stopped state (default), the increment of the edit substate pushes the state into set. However if the start button is pressed then the state will change to running. From the running state, pressing the start button will push the state into stopped. From the set state, the edit substate returning to 000 will push the system back to the stopped state.

0.5.14 Report 14

The exact specification for each model and exact testing for each model meant I had certainty that my previous work would hold up to future scrutiny in more complex modules. It also meant that my code itself was documentation and required only a small portion of written documentation. This made it easy to debug and test modules later down the line as I found issues in parent modules. However, the test were often overly specific, and assumed a very specific architecture which was not necessarily the only way to complete a task. This caused much frustration and time loss. In future, I would take on the task of writing tests to fit a simpler solution, or a solution that is more obvious to myself.

0.5.15 Report 15

Given this project took me 30 hours to complete, I would say VS code and modern tools saved me 10 hours. The linting features were a life saver and likely saved me 4 hours, as it would take me 3 or 4 lints to remove all errors and warnings from each module. Automated tests saved me anywhere from 10 to 20 hours as I could have certainty that the code I had written was mostly correct, and would not cause an error during the testing of later modules. The same can be said for formal verification. Without these tools, the problems I encountered would have remained hidden until the top modules of each section, where they would have been impossible to debug within the time constraints.

0.5.16 Report 16

The \$lt cell corresponds to a less than operator, used in the comparison if statement that determines whether to reset or continue counting up. The \$add cell corresponds to the addition statement in the count case of the if statement ($\text{next_count} = \text{count} + 1$). Finally, the \$mux represents a multiplexer that switches between A and B (reset and counting modes) using the result of the comparison.

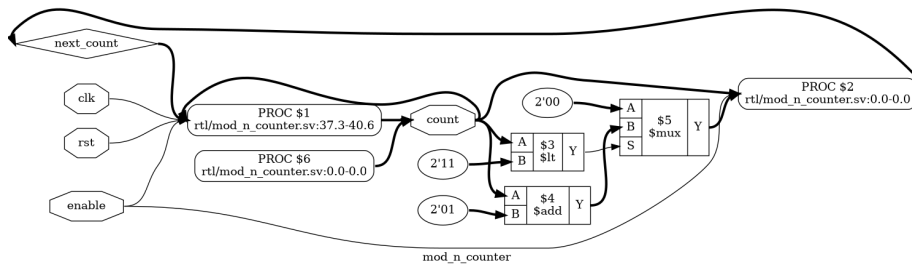


Figure 24: Elaborated diagram produced by yosys.

0.5.17 Report 17

\$sdffe is the yosys block for a flip flop with synchronous reset and an enable pin. The CLK pin is the system clock input, the D pin is the input, which is driven by the next state logic (as seen with the arrow), the EN input is the enable signal and SRST is the synchronous reset. The proc command appears to have kept combinational logic untouched, but has combined PROC cells into appropriate flip flops.

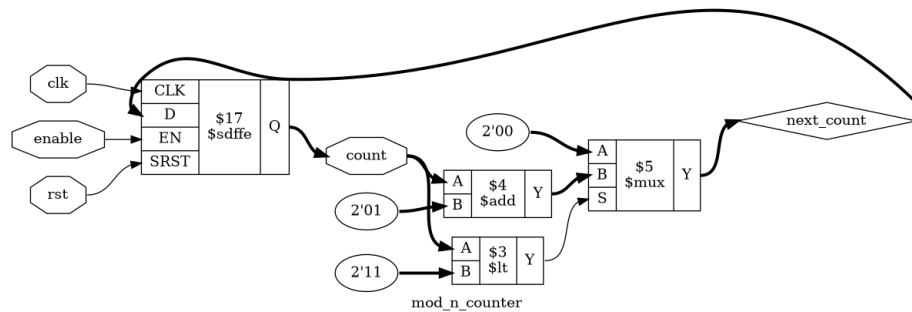


Figure 25: Proc diagram produced by yosys.

0.5.18 Report 18

It is possible that the doubling of flip flops is a result of the initialisation of the count variable. Yosys requires this logic split to move the correct logic signals to the correct logic gates. Unlike the cells, it is normal for signals from a register to enter a set of logic gates in a non sequential order. The PPOP configures four polarity values, the clock trigger (P:positive), reset (P:positive), reset value (O:resets to 0) and the enable (P:active high).

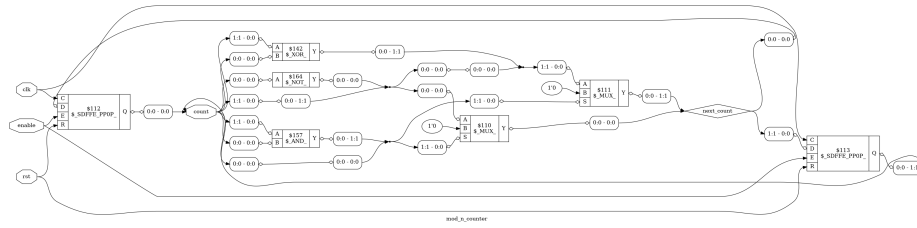


Figure 26: Techmap diagram produced by yosys.

0.5.19 Report 19

REQUIRES DIAGRAM, LEFT FOR FINAL RUN OVER

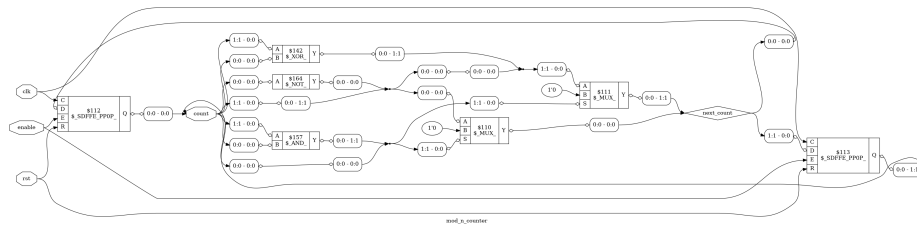


Figure 27: Abc diagram produced by yosys.

0.5.20 Report 20

No extra features were added. Time was focused on testing and debugging.

0.5.21 Report 21

Basic clocking, rising edge detection and counting is the ideal use case for digital logic on an FPGA. However,